Input –

```
from scapy.all import sniff, Dot11
import time
import subprocess

INTERFACE = "wlan0mon"
AP_BSSID = "AA:BB:CC:DD:EE:FF"      # MAC of our Access Point
THRESHOLD = 10                  # auth attempts
TIME_WINDOW = 15                  # seconds
BLOCKED_MACS = set()

auth_attempts = {}   # { mac : [timestamps] }

def block_mac(mac):
   """Add attacker MAC to hostapd deny list and reload."""
   if mac in BLOCKED_MACS:
      return
   BLOCKED_MACS.add(mac)
   print(f"[ALERT] Blocking suspected attacker MAC: {mac}")
   with open("/etc/hostapd/hostapd.deny", "a") as f:
      f.write(mac + "\n")
   subprocess.run(["sudo", "systemctl", "reload", "hostapd"])
   # Also drop at firewall level for wired segment (printer protection)
   subprocess.run(["sudo", "iptables", "-A", "INPUT",
                "-m", "mac", "--mac-source", mac, "-j", "DROP"])

def detect_cracking(pkt):
   if pkt.haslayer(Dot11):
      if pkt.type == 0 and pkt.subtype == 11:   # Management / Authentication
         src = pkt.addr2
         dst = pkt.addr1
         if dst != AP_BSSID or src is None:
            return

         now = time.time()
         auth_attempts.setdefault(src, [])
         auth_attempts[src].append(now)

         # keep only attempts within TIME_WINDOW
         auth_attempts[src] = [t for t in auth_attempts[src]
                           if now - t <= TIME_WINDOW]
```

```
        count = len(auth_attempts[src])
        print(f"[INFO] Auth attempt #{count} from {src} -> AP {dst}")

        if count >= THRESHOLD:
            print(f"[WARNING] Possible WPA/WEP password-cracking attack "
                f"from {src} ({count} attempts in {TIME_WINDOW}s)")
            block_mac(src)

print(f"[*] Monitoring {INTERFACE} for WiFi crack attempts targeting {AP_BSSID} ...")
sniff(iface=INTERFACE, prn=detect_cracking, store=False)
```

output –

